



Memory: From the Abstract Machine to Smart Pointers

Elias Farhan

University of
Hertfordshire **UH**



What are we doing today?

- The C++ **abstract machine** and its memory model
- Where objects actually live: **stack, heap, static**
- A quick word on **thread_local**
- **Smart pointers**: `unique_ptr`, `shared_ptr`, `weak_ptr`
- A peek under new: an **introduction to allocation**



The abstract machine



C++ does not target your CPU

The standard never mentions your specific processor, OS, or RAM.

- It describes a hypothetical **abstract machine**
- Your code defines its **observable behavior**
- The compiler must reproduce that behavior, however it likes



The as-if rule

The compiler may do **anything**, as long as the program's observable behavior is preserved.

- Reorder, fuse, or delete operations
- Keep a value in a register, never touching RAM
- Skip an allocation entirely

```
int x = 2 + 3;           // may be compiled as: int x = 5;  
int y = x * 0;          // may be compiled as: int y = 0;
```



Variables vs. objects

We say “variable”, the standard says **object**: a region of storage with a type.

- A **type** (how to interpret the bytes)
- A **value** (the bytes themselves)
- A **size** and **alignment**
- A **storage duration** and a **lifetime**



Object lifetime

An object exists only between two precise points.

- **Begins:** storage is obtained *and* initialization completes
- **Ends:** the destructor runs *and* storage is released
- Touching it outside that window is **undefined behavior**

```
int* p;  
{  
    int local = 42;  
    p = &local;           // ok, local is alive  
                           // UB: local's lifetime ended  
} *p;
```



Storage duration

The model defines **four** flavors of storage. They decide *when* an object lives.

Duration	Lives for...	Created by
automatic	the enclosing scope	local variables
static	the whole program	globals, static locals
dynamic	until you free it	new / allocators
thread	the owning thread	thread_local



What if there was a way...

to see *where* the abstract machine's storage durations land in a **real** running process?

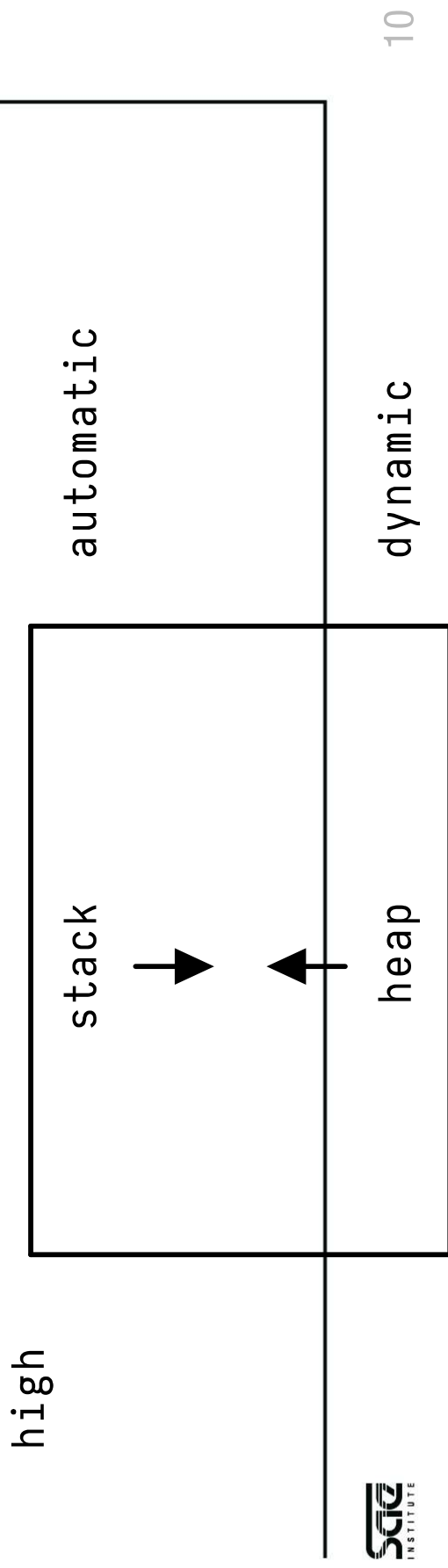


Where memory actually lives



The process address space

One typical layout, low address at the bottom.





The stack

Automatic storage. A region that grows and shrinks with **function calls**.

- Each call pushes a **frame**: parameters, locals, return address
- Returning **pops** the frame, instantly
- Just a pointer move: allocation is essentially **free**

```
void f() {  
    int a = 1;           // pushed on entry  
    double b = 2.0;      // pushed on entry  
                        // both popped on return  
}
```



The stack is fast but small

Speed comes from its strict **LIFO** discipline and cache-friendliness.

- Typical size: a few **MB** per thread (OS-defined)
- Lifetime is **tied to scope**, not to your wishes
- Too deep, or too big → **stack overflow**

```
void recurse() { recurse(); } // crash: no base case  
int big[10'000'000]; // crash: blows the stack frame
```



The heap

Dynamic storage. Memory you request **explicitly** and keep as long as you need.

- Lifetime is **decoupled** from scope
- Sized at **runtime**, can be huge
- You (or a smart pointer) are responsible for **releasing** it

```
int* p = new int{42}; // allocate on the heap
// ... use p ...
delete p; // you must give it back
```



The heap is flexible but costly

Every allocation talks to an **allocator** that manages free blocks.

- Far slower than a stack push (bookkeeping, possibly a syscall)
- Can **fragment**: free memory exists but not in one block
- Forget delete → **leak**; delete twice → **corruption**



Static storage

One instance, born before `main`, alive until the program exits.

- File-scope globals, static members, function static locals
- Zero-initialized first, then any constant/dynamic init
- A function static is initialized **once**, on first reach

```
int counter = 0;           // static duration

int next_id() {
    static int id = 0;      // initialized once, survives calls
    return ++id;
}
```



static: same word, different jobs

The keyword does several unrelated things depending on **where** it sits.

- Inside a function → one instance kept across calls (*seen above*)
- At **file scope** → it controls **linkage** (this section)
- On a **class member** → one instance shared by all objects



Translation units & linkage

The compiler sees one **translation unit** (a .cpp plus its headers) at a time; the linker then joins them.

- **Linkage** = can this name be referred to from *another* TU?
- **External**: one shared entity across the whole program
- **Internal**: private to its own TU



Globals have external linkage

A plain namespace-scope variable or function is visible to **every** TU.

```
// config.cpp
int g_volume = 100;           // external linkage: one global

// other.cpp
extern int g_volume;          // refers to the SAME object
```



static = internal linkage

Put static on a file-scope name and it becomes **private** to its TU.

```
// audio.cpp  
static int g_volume = 100;      // internal: this TU only  
  
// other.cpp  
static int g_volume = 50;      // a DIFFERENT, independent object
```



The anonymous namespace

The modern replacement. Same internal linkage, but for **any** entity, even types.

```
namespace {  
    int g_volume = 100;           // internal linkage  
    void reset() { g_volume = 0; } // internal linkage  
    struct Cache { /* ... */ };  // a type too: static can't do this  
}
```



static vs anonymous namespace

Both give internal linkage; the namespace is more capable and is the C++ recommendation.

	file-scope static	anonymous namespace
linkage	internal	internal
variables, functions	✓	✓
types	X	✓
guideline	legacy C style	preferred



The One Definition Rule (ODR)

An external-linkage entity may have **exactly one** definition in the whole program.

- Many **declarations** are fine; only one **definition**
- Violations are often **silent**: linker errors, or worse, UB
- Internal linkage sidesteps it: each TU owns its own copy



The header global trap

Define a global in a header, and every TU that includes it defines it **again**.

```
// settings.h  -- included by many .cpp files
int          g_count = 0;           // BAD: multiple definitions → linker error
inline int g_count = 0;           // C++17: one shared definition, header-safe
extern int g_count;                // declaration only; define once in a .cpp
namespace { int g_count = 0; }     // internal: a private copy per TU
```



static class members

One instance shared by **all** objects of the class, not one per object.

```
struct Widget {  
    static int s_count;           // declaration: external linkage  
    static void reset();         // shared, has no `this`  
};  
  
int Widget::s_count = 0;         // definition: exactly once, in a .cpp
```



static class members the modern way

One instance shared by **all** objects of the class, not one per object.

```
struct Widget {  
    inline static int s_count = 0;    // since C++17  
    static void reset();             // shared, has no `this`  
};
```



thread_local

Like static, but **one copy per thread**, not one for the whole program.

- Each thread gets its own independent instance
- Born when the thread starts, dies when it ends
- No sharing → no data race on that variable

```
thread_local int errors = 0; // every thread counts its own
```



What if there was a way...

to get the heap's **flexibility** without ever forgetting to call `delete`?



Smart pointers



Manual memory is a trap

A single owning raw pointer invites three classic bugs.

```
Widget* w = new Widget{};
if (early_exit) return;           // LEAK: delete never runs
use(*w);
delete w;                          // DOUBLE FREE: corruption
delete w;
```



RAII, again

Tie the heap allocation to a **stack object**. Its destructor frees, always.

- Acquire in the constructor, release in the destructor
- Works on early return, on exceptions, everywhere
- The standard library ships ready-made owners: **smart pointers**



unique_ptr: exclusive ownership

Exactly **one** owner. Move-only, so ownership can be transferred but never shared.

```
#include <memory>

std::unique_ptr< Widget > a = std::make_unique< Widget >();
std::unique_ptr< Widget > b = std::move(a);    // ownership moves, a is now empty
// auto c = b;                                // ERROR: cannot copy
// b deletes the Widget at end of scope
```



unique_ptr: zero overhead

The same size and speed as a raw pointer, with automatic cleanup for free.

- No reference count, no bookkeeping
- Prefer `make_unique`: exception-safe, no naked new
- This is your **default** owning pointer

```
auto w = std::make_unique< Widget >(args...); // one heap object, one owner
```



shared_ptr: shared ownership

Many owners, **one** object. A reference count tracks how many are left.

```
auto a = std::make_shared< Widget >(); // count = 1
auto b = a;                          // count = 2
a.reset();                           // count = 1
// last shared_ptr to die deletes the Widget (count → 0)
```



shared_ptr: the cost

Sharing is not free. A shared_ptr is **two** pointers plus a control block.

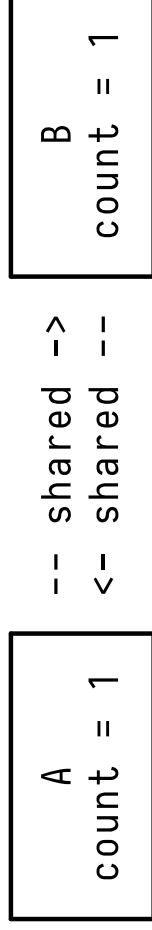
- A **control block**: strong count, weak count, deleter
- Counts are **atomic** → thread-safe, but slower to copy
- make_shared fuses object + control block into **one** allocation

```
auto w = std::make_shared<Widget>(); // one allocation instead of two
```



Cycles leak

Two shared_ptrs pointing at each other keep the count **stuck above zero**.



nobody else owns them, yet neither is freed



weak_ptr: observe without owning

A non-owning reference to a shared_ptr's object. Does **not** keep it alive.

- Breaks reference cycles (child points back to parent weakly)
- lock() returns a shared_ptr, or empty if the object is gone
- The safe way to ask “is it still there?”

```
std::weak_ptr< Widget > obs = shared;  
if (auto s = obs.lock()) { use(*s); } // got a temporary owner  
else { /* gone */ }
```



Which pointer when?

Reach for the cheapest tool that expresses your ownership.

Need	Use
one owner (the default)	<code>unique_ptr</code>
genuinely shared ownership	<code>shared_ptr</code>
observe, don't own	<code>weak_ptr</code>
borrow, no ownership at all	<code>raw *</code> / <code>&</code>



What if there was a way...

to see what new and make_unique are **actually** doing underneath?



A peek under new



new is two steps

new is not one operation. It **allocates**, then **constructs**.

```
Widget* w = new Widget{args};  
  
// conceptually:  
void* mem = operator new(sizeof(Widget)); // 1. get raw bytes  
Widget* w = new (mem) Widget{args}; // 2. construct in place
```



delete is two steps too

Symmetrically, `delete` destroys, then deallocates.

```
delete w;  
  
// conceptually:  
w→~Widget();           // 1. run the destructor  
operator delete(w);      // 2. give the bytes back
```



References

- [cppreference: object model](#)
- [cppreference: storage duration](#)
- [cppreference: smart pointers](#)
- [cppreference: new expression](#)
- Scott Meyers, *Effective Modern C++*, items 18-22 (smart pointers)
- “What Programmers Should Know About Memory”, Ulrich Drepper



Conclusion

- The **abstract machine** defines behavior; the **as-if rule** lets the compiler optimize freely.
- Storage duration decides lifetime: **automatic** (stack), **static**, **dynamic** (heap), **thread**.
- The **stack** is fast but small; the **heap** is flexible but costly.
- File-scope **static** and the **anonymous namespace** give *internal linkage* (TU-local), unlike a plain global; the namespace is preferred.
- **Smart pointers** are RAII for the heap: `unique_ptr` by default, `shared_ptr` when truly shared, `weak_ptr` to observe.
- `new` = `allocate` + `construct`. Split them with **placement new**, and you can build your